Raw decoder excerpt (continued) *preprocessing/pingstation_raw_decoder.py*

```
75
76      def log_decoded_many(self, records):
77          for record in records or []:
78              self.log_decoded(record)
79
80
81  class PingstationRawDecoder:
82      """
83      Decodes Pingstation length-prefixed binary frames into structured aircraft records.
84      """
85
86      def __init__(self):
87          self._state = {}
88
89      # Compressed VRS field flags.
90      _F_CALLSIGN = 0x0001
91      _F_ALTITUDE = 0x0002
92      _F_GROUNDSPEED = 0x0004
93      _F_TRACK = 0x0008
94      _F_LAT = 0x0010
95      _F_LON = 0x0020
96      _F_VSPEED = 0x0040
97      _F_SQUAWK = 0x0080
98      _F_COMP_FLAGS = 0x0F00
99
100     def reset_state(self):
101         self._state.clear()
102
103     def split_frames(self, buffer_bytes):
104         """
105         Splits a byte buffer into complete frames using the first byte as frame length.
106         Returns (frames, remainder).
107         """
108         frames = []
109         idx = 0
110         total = len(buffer_bytes)
111
112         while idx < total:
113             frame_len = buffer_bytes[idx]
114
115             # Discard obvious noise and re-sync.
116             if frame_len < 10 or frame_len > 120:
117                 idx += 1
118                 continue
119
120             end_idx = idx + frame_len
121             if end_idx > total:
122                 break
123
124             frames.append(buffer_bytes[idx:end_idx])
125             idx = end_idx
126
127         return frames, buffer_bytes[idx:]
128
129     def _decode_callsign(self, frame_bytes):
130         if len(frame_bytes) >= 18:
131             field = frame_bytes[10:18]
132             text = field.decode("ascii", errors="ignore").strip().upper()
133             text = re.sub(r"[^A-Z0-9]", "", text)
134             if text:
135                 return text
136
137         # Fallback when fixed field is absent/corrupt.
138         tokens = re.findall(rb"[A-Z0-9]{3,8}", frame_bytes)
139         for token in tokens:
140             candidate = token.decode("ascii", errors="ignore").upper()
141             if any(ch.isalpha() for ch in candidate):
142                 return candidate
143
144         return "UNKNOWN"
145
146     def _read_float_int24(self, frame_bytes, idx):
147         if idx + 3 > len(frame_bytes):
148             return None, idx
```